CHARUSAT®
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 10: Hash Tables: Linear Probing, Chaining, and Double Hashing

Practical Aim:

To implement hash tables using different collision handling strategies and develop the ability to reason about why collisions occur, how different strategies resolve them, and what trade-offs each approach involves.

Problem 1 :

A small parking lot has exactly 10 numbered slots (0 to 9). When a vehicle arrives, its registration number is used to assign it a slot by taking the last digit of the number. If that slot is already taken, the vehicle moves to the next available slot in order, wrapping around if necessary. Given a sequence of vehicle registrations, assign each to a slot using this rule and display the final state of all ten slots.

Describe what your solution does when it encounters an already occupied slot. What happens when the parking lot is nearly full and many vehicles keep landing on the same slot? Can you think of a situation where your solution might loop indefinitely?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-10

Output:

```
PS C:\Users\ADMIN\OneDrive\Desktop\DSA> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-10\" ; if ($?) { g++ PR
A10-1.cpp -o PRA10-1 } ; if ($?) { .\PRA10-1 }
Enter number of vehicles: 6

 enter youe registration number : 101

 enter youe registration number : 102

 enter youe registration number : 210

 enter youe registration number : 112

 enter youe registration number : 111

 enter youe registration number : 311

Final Parking State:
Slot 0 : 210
Slot 1 : 101
Slot 2 : 102
Slot 3 : 112
Slot 4 : 111
Slot 5 : 311
Slot 6 : Empty
Slot 7 : Empty
Slot 8 : Empty
Slot 9 : Empty
PS C:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-10>
```

Problem Faced:

The main challenge was implementing the wrap-around probing correctly using the modulo operator and making sure the program does not loop forever when the parking lot is completely full, since in that case every slot would eventually be checked and found occupied without a free slot ever being found.

Outcome/Conclusion:

Each vehicle was assigned an initial slot equal to the last digit of its registration number. If that slot was already occupied, the program checked the next slot in order, wrapping back to slot 0 after slot 9 using the modulo operator, until a free slot was found. The final occupancy of all ten slots was then displayed.

When an already occupied slot is encountered, the solution simply advances to the next slot (slot = (slot + 1) % 10) and checks again, repeating this until an empty slot is found. When the lot is nearly full and many vehicles keep landing on the same digit, this causes a cluster of consecutive occupied slots, and each new colliding vehicle has to probe through the whole cluster before finding a free slot, making insertions progressively slower. If the parking lot becomes completely full and a solution does not keep a count of how many vehicles have already been placed, the probing would check all 10 slots and return to the starting slot without ever finding a free one, looping indefinitely; this was avoided by tracking the number of vehicles already parked and stopping with an error once that count reaches 10.

Problem 2 :

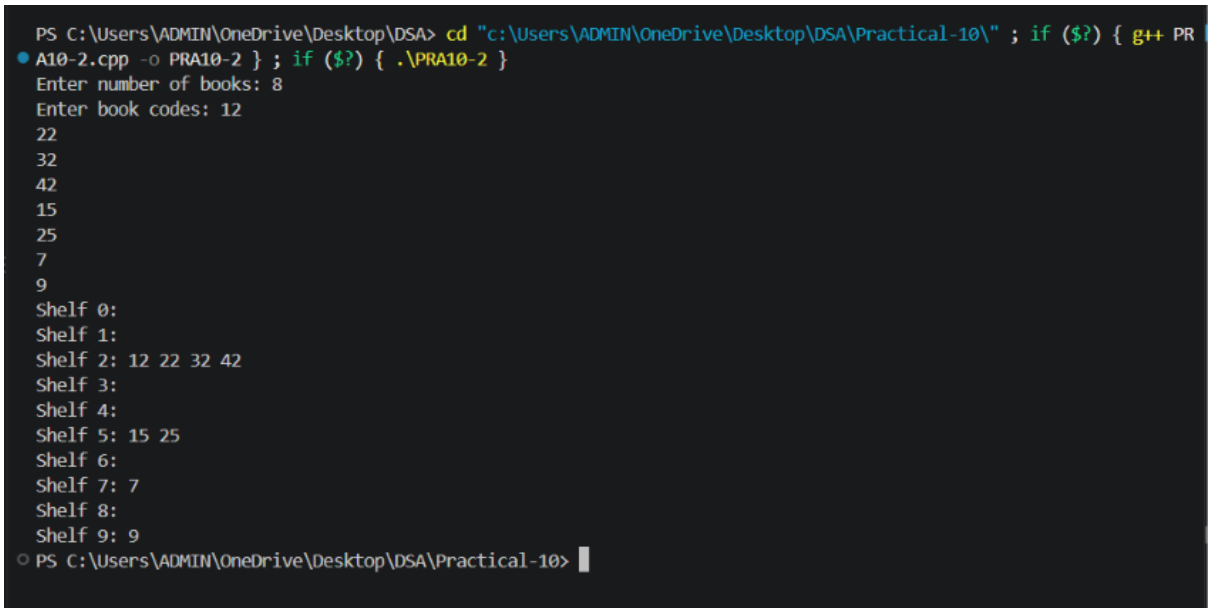
A library uses a system with 10 shelves numbered 0 to 9. Each book's code is used to decide which shelf it belongs to by taking the code modulo 10. Unlike the parking lot, each shelf can hold multiple books stacked together — when two books map to the same shelf, they are simply added to that shelf's stack. Given a sequence of book codes, place each on the correct shelf and display the final contents of all shelves.

Describe how your solution handles two books that map to the same shelf. What happens to search time on a shelf as more and more books pile onto it? Can you think of what would cause all books to end up on the same shelf?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-10

Output:



Problem Faced:

The main challenge was implementing chaining using a linked list at every shelf position (an array of linked lists), making sure new books are added to the correct shelf's chain without disturbing the books already there, and printing the contents of every shelf, including shelves that remain empty.

Outcome/Conclusion:

Each shelf was represented as a linked list, and the library was represented as an array of 10 such lists. Every book code was placed on the shelf given by code modulo 10, and if a book already existed on that shelf, the new book was simply added to that shelf's linked list. The final contents of all ten shelves were then displayed.

When two books map to the same shelf, the solution does not displace or move any existing book; it simply appends (or inserts) the new book into that shelf's linked list, so a shelf with no collisions holds one book while a shelf with several colliding codes holds all of them in its chain. As more books pile onto a shelf, searching that shelf takes longer because the chain must be traversed one node at a time, so search time on a heavily loaded shelf grows roughly in proportion to the number of books chained there, approaching the slowness of a plain linear search in the worst case. All books would end up on the same shelf if every incoming book code gave the same remainder when divided by 10 — for example, if every code were a multiple of 10, all of them would map to shelf 0, leaving the other nine shelves completely empty.

Problem 3 :

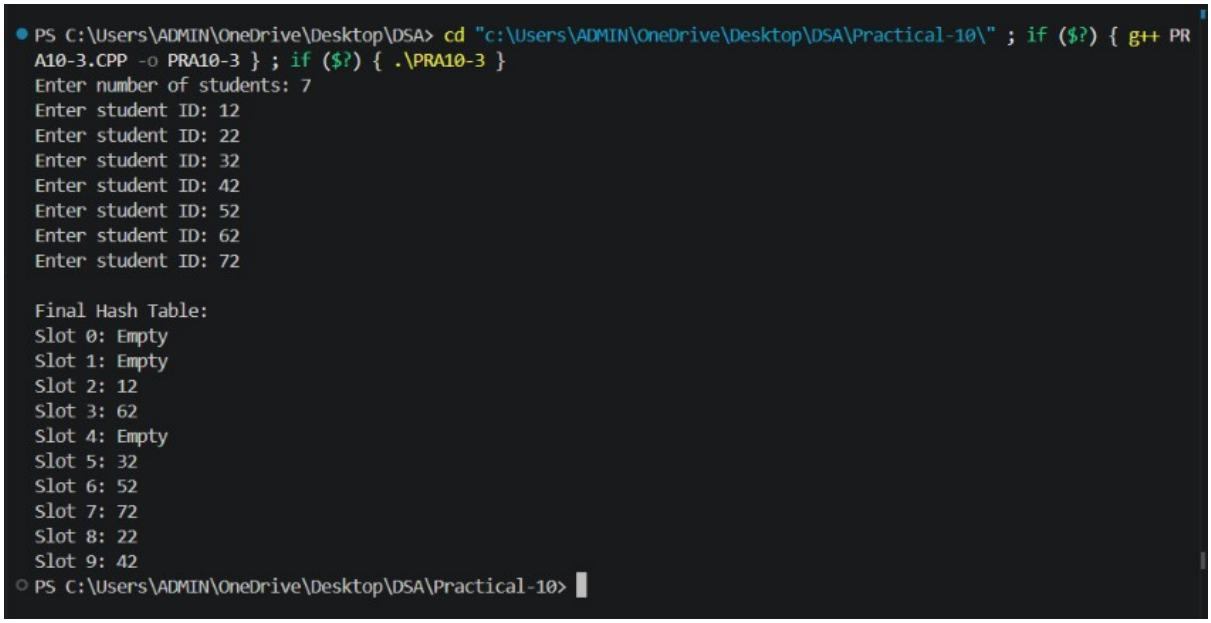
A university assigns student IDs to storage slots using a two-step rule. The first rule determines the initial slot. If that slot is taken, instead of simply moving one slot forward, a second rule based on the ID itself determines how many slots to jump. This means different IDs that collide will probe different sequences of slots, spreading them more evenly. Given a sequence of student IDs, assign each to a slot using both rules and display the final state of the table.

Describe how your solution decides how many slots to jump when a collision occurs. What is the risk if the jump size happens to be a factor of the table size? How does this strategy differ from Problem 10.1 in how it spreads collisions?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-10

Output:



Problem Faced:

The main challenge was choosing a second hash function that produces a usable jump size for every ID, making sure it never produces a jump of zero (which

would stop the probe sequence from moving at all), and verifying that the probing still terminates correctly either by finding a free slot or by recognising that the table is full.

Outcome/Conclusion:

Each student ID was first assigned an initial slot using the first rule (ID modulo the table size). If that slot was occupied, a second rule based on the same ID (such as a fixed value minus the ID modulo that value, chosen to always give a non-zero result) determined the jump size, and the program kept adding that jump, wrapping around the table, until a free slot was found. The final state of the table was then displayed.

The jump size for a colliding ID is computed once, from the ID itself, using the second hash rule, so two different colliding IDs generally get two different jump sizes and therefore follow different probe sequences around the table, rather than all following the same path. The risk if the jump size happens to be a factor of the table size is that the probe sequence will only ever visit a limited subset of the slots (those at multiples of that factor from the starting slot) and cycle through just that subset forever, potentially failing to find a free slot even though other, unreachable slots are still empty. This differs from Problem 1's linear probing, where every colliding key always moves forward by exactly one slot regardless of its value, which tends to create long clusters of adjacent occupied slots; double hashing spreads collisions more evenly across the table because the step size itself varies from key to key.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation:

[Problem 1] If five consecutive vehicle registrations all end in the same digit, trace where each one gets placed in the parking lot. Now explain why this clustering effect is a structural weakness of this collision strategy — not just for this input, but in general.

Sol: If five consecutive registrations all end in the same digit d, the first vehicle takes slot d, the second finds d occupied and takes slot (d+1)%10, the third finds

both d and (d+1)%10 occupied and takes (d+2)%10, and so on, so all five vehicles end up occupying five consecutive slots starting at d. This is a structural weakness of linear probing in general, not just for this input, because the fixed "move forward by one" rule always extends an existing block of occupied slots by exactly one more slot whenever a new collision happens at that block's edge; this means any sequence of colliding keys, not only this particular one, will always grow into one unbroken cluster rather than spreading out, and once a cluster exists, it also makes future unrelated keys that happen to land near it more likely to collide with the cluster too, which is known as primary clustering.

[Problem 2] Suppose all book codes in Problem 2 are multiples of 10. Without tracing through the code, predict what the final shelf layout would look like and explain why.

Sol: If every book code is a multiple of 10, then every code modulo 10 equals 0, so every single book would be hashed to shelf 0, which would end up holding all the books chained together, while shelves 1 through 9 would remain completely empty. This happens because the hash function (code modulo 10) only looks at the remainder after dividing by 10, and a number that is an exact multiple of 10 always has a remainder of 0; since the input data in this case only ever produces that one remainder, the hash function cannot distribute it across the other shelves no matter how many books arrive. This shows that a hash function's ability to spread data evenly depends not only on the function itself but also on whether the actual input values vary enough across the range the function depends on — a function that works well for varied, general input can still fail badly on input that shares a common structural pattern.

[Problem 3] In Problem 3, what would happen if the second rule always produced a jump size of zero? Trace through the consequence and explain how this failure is related to the choice of the second hash function rather than the data itself.

Sol: If the second rule always produced a jump size of zero, then whenever a collision occurred, the probe sequence would keep adding zero to the current slot, so it would check the exact same occupied slot over and over again and never move to any other position. The program would therefore either loop forever re-checking the same slot, or, if it has a step limit, incorrectly report the

table as full even when other slots are empty, since it never actually reaches them. This failure is entirely caused by the choice of the second hash function rather than by the student IDs themselves: no matter what IDs are supplied, a second hash function that can return zero breaks the probing mechanism for every key that happens to collide, which is why a correct double hashing implementation must be designed so its second hash function can never produce a zero step (for example, by using a formula such as a fixed value minus the ID modulo that value, with the fixed value chosen so the result is always at least one).